

Unit-01 Introduction to .NET

1.1 .NET Framework: Features and Architecture

Features of .NET Framework

- **Language Interoperability:** Supports multiple languages (C#, VB.NET, F#).
- **Automatic Memory Management:** Uses garbage collection for memory optimization.
- **Security:** Provides Code Access Security (CAS) and role-based authentication.
- **Rich Standard Library:** Pre-built libraries for various functionalities.
- **Cross-Platform Development:** .NET Core allows development on Windows, Linux, and macOS.
- **Scalability and Performance:** Optimized for high-performance applications.
- **Integrated Development Environment (IDE) Support:** Works with Visual Studio for efficient coding.
- **Support for Web, Desktop, and Mobile Development:** Single framework for diverse applications.

Architecture of .NET Framework

1. **Common Language Runtime (CLR):** Executes .NET applications, manages memory, and handles exceptions.
2. **.NET Framework Class Library (FCL):** Collection of reusable classes and APIs.
3. **Common Type System (CTS):** Defines standard data types across .NET languages.
4. **Metadata and Assemblies:** Provides information about compiled code.
5. **Windows Forms and ASP.NET:** For developing desktop and web applications.
6. **Interoperability:** Allows interaction with COM objects and unmanaged code.

1.2 .NET Components

Common Language Runtime (CLR)

- Manages program execution.
- Handles garbage collection and exception handling.
- Provides Just-In-Time (JIT) compilation.
- Ensures type safety and security enforcement.

Class Library

- Collection of pre-built classes and methods.
- Supports database connectivity, file handling, and network communication.
- Includes namespaces such as System, System.IO, System.Data, etc.

1.3 .NET Framework, .NET Core, and .NET Standard

Feature	.NET Framework	.NET Core	.NET Standard
Platform	Windows Only	Cross-Platform	API Specification
Performance	Moderate	High	-
Open-Source	No	Yes	Yes
Compatibility	Older applications	Modern apps	Ensures consistency
Deployment	System-wide	Self-contained	-
Microservices Support	Limited	Full	-

1.4 Introduction to Visual Studio and Visual Studio Code IDE

Visual Studio

- Full-featured IDE for .NET development.
- Supports debugging, IntelliSense, and Git integration.
- Includes features like CodeLens, Live Share, and performance profiling.

Visual Studio Code

- Lightweight, cross-platform editor.
- Extensions available for .NET development.
- Supports integrated terminal, debugging, and Git control.

Setting Up Visual Studio Development Environment

1. Download and install Visual Studio from Microsoft.
2. Select the required workloads (ASP.NET, .NET Core, etc.).
3. Configure project settings and start coding.
4. Install necessary extensions for Visual Studio Code if using VS Code.

IntelliSense

- Provides code suggestions and auto-completion.
- Enhances productivity and reduces syntax errors.
- Helps with method signatures, variables, and object properties.

1.5 Project Types in .NET

- **Console Applications:** Command-line applications.
- **Windows Forms Applications:** GUI-based desktop applications.
- **ASP.NET Web Applications:** Web development using .NET.
- **Blazor Applications:** Web applications using C# and WebAssembly.
- **Class Libraries:** Reusable code libraries.
- **Mobile Applications:** Xamarin for Android and iOS apps.
- **Microservices Applications:** Distributed architecture support.
- **Cloud-Based Applications:** Azure integration for cloud-native applications.